3D WEBSOFT INC. FOUNDATION

FUTURE TECH SCHOOL PROGRAM

Faculty Name :

Course Duration : 5 Months

Classes : Offline & Online

Daily Teaching : 2 Hours

Total Classes : 100

Level 4 – AI Professional Foundation(11th–12th Standard)

Class 13 – Modules & Packages in Python

Modules and Packages

Module: Smallest unit of code reusability

File containing Python function definitions

Package: Logical collection of modules

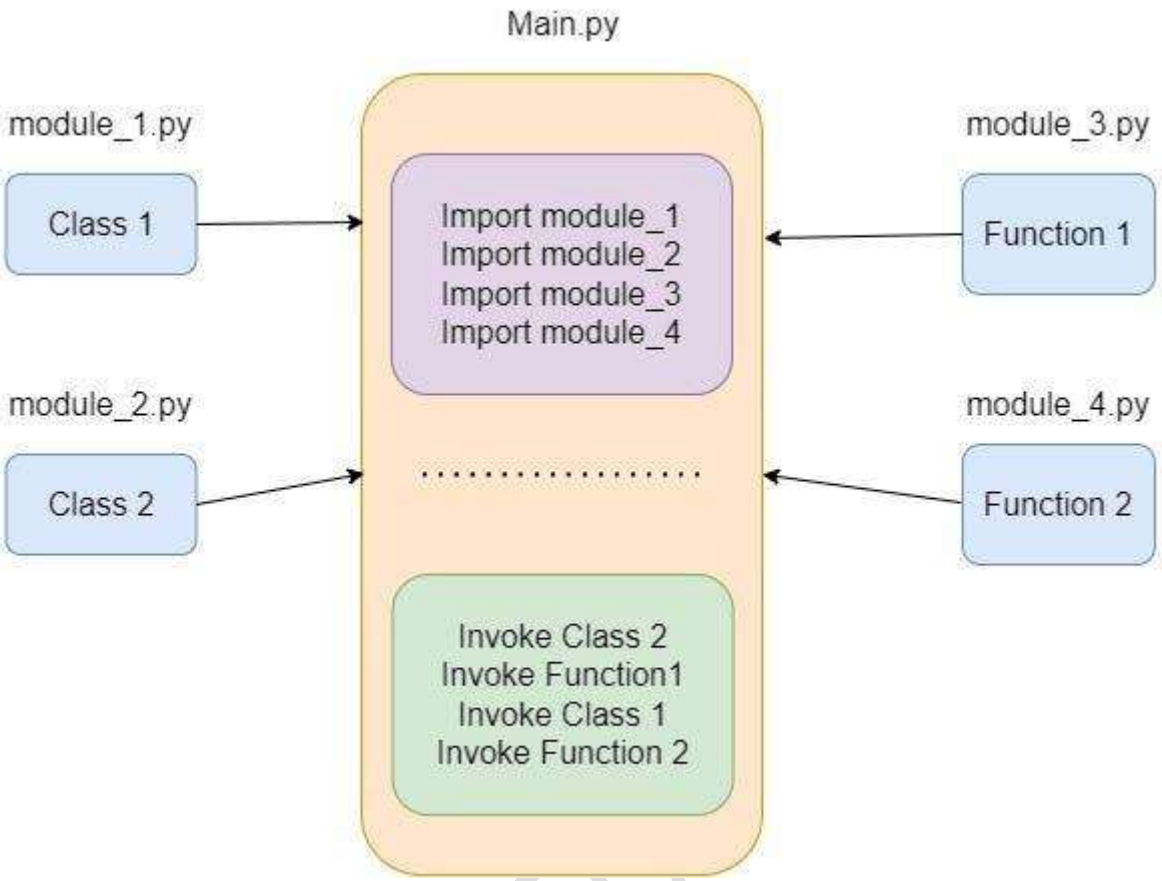
Often bundles large products and broad functionality

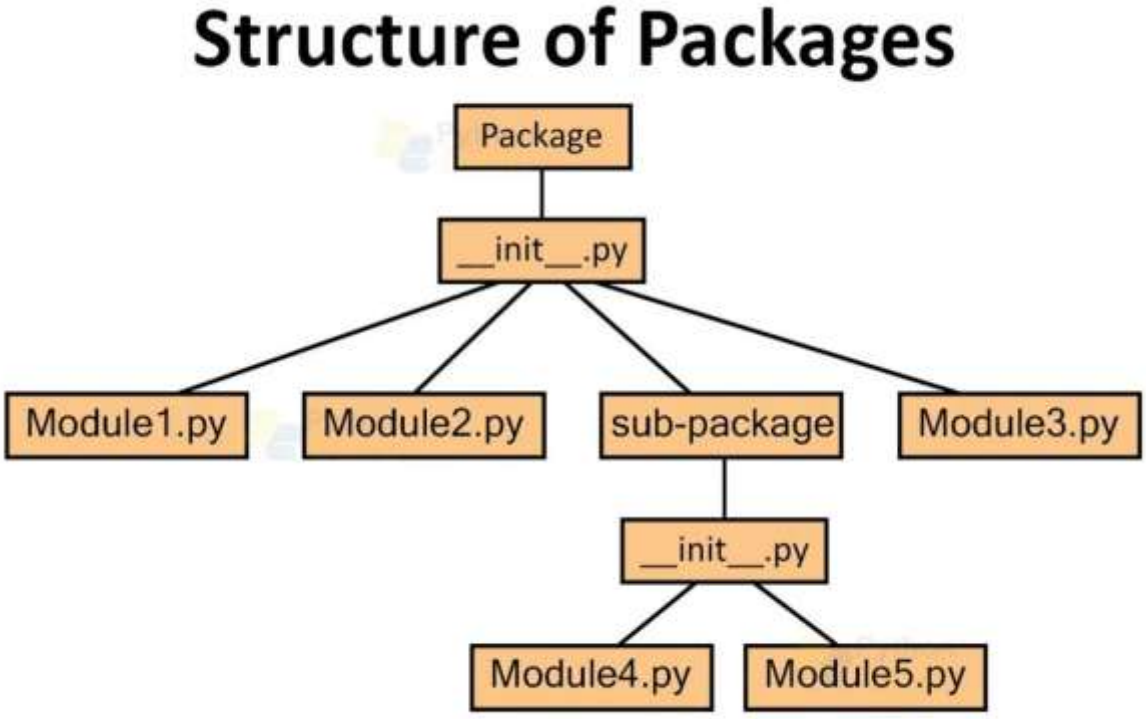
Library: Collection of packages and modules



Dr.S.Sangeetha, Department of CA, NIT Trichy

FOUNDATION





1. Learning Objectives

By the end of this lesson, students will be able to:

- Explain what a Python module is.
- Understand why modules are useful.
- Import built-in modules.
- Use functions from modules.
- Create a **custom module**.
- Import a custom module into another Python program.
- Understand Python packages.
- Create a simple custom package.
- Organize large programs into multiple files.

2. What is a Module?

A **module** is a Python file containing reusable Python code.

A module can contain:

- Variables
- Functions
- Classes
- Statements

A Python module normally has the extension:

.py

Example

Suppose we create:

calculator.py

3DWEBSOFT FOUNDATION

Inside it:

```
def add(a, b):
    return a + b


def subtract(a, b):
    return a - b
```

We can use these functions from another Python program.

3. Why Do We Need Modules?

Imagine writing one huge Python program:

```
student_management.py
        ↓
5000 lines of code
        ↓
Difficult to understand
Difficult to maintain
Difficult to reuse
```

Instead, divide it:

```
Main Program
     |
     ├── student.py
     ├── marks.py
     ├── fees.py
```

3UWEBSOFT FOUNDATION

└── reports.py

This makes programs:

- Organized
- Reusable
- Easier to debug
- Easier to maintain
- Easier to understand

4. Real-Life Example

Think about a **school bag**.

Instead of keeping everything loose:

Books

Pens

Pencils

Geometry box

Notes

we organize them into different sections.

Python modules work similarly.

Large Program

|

├── Module 1

├── Module 2

├── Module 3

3DWEBSOFT FOUNDATION

└── Module 4

Each module handles a particular task.

5. Types of Python Modules

There are three important categories for students to understand.

1. Built-in / Standard Library Modules

These come with Python.

Examples:

import math

import random

import datetime

2. Third-Party Modules

These are installed separately.

Examples include libraries such as:

requests

numpy

pandas

3. Custom Modules

These are modules **created by the programmer**.

Example:

calculator.py

student.py

marks.py

3DWEBSOFT FOUNDATION

6. Importing a Module

The import statement allows us to use a module.

Syntax

import module_name

Example

import math

print(math.sqrt(25))

Output:

5.0

Here:

math

↓

Module

sqrt()

↓

Function inside the module

7. Using math Module

import math

print(math.sqrt(64))

3DWEBSOFT FOUNDATION

print(math.pi)

print(math.factorial(5))

Output:

8.0

3.141592653589793

120

8. Importing Specific Functions

Instead of importing the complete module:

import math

we can import a specific function.

from math import sqrt

print(sqrt(49))

Output:

7.0

Syntax

from module_name import function_name

9. Import Multiple Functions

from math import sqrt, factorial

print(sqrt(36))

3DWEBSOFT FOUNDATION

print(factorial(5))

Output:

6.0

120

10. Using an Alias

We can give a shorter name to a module.

import math as m

print(m.sqrt(81))

Output:

9.0

Here:

math → Original name

m → Alias

11. What is a Custom Module?

A **custom module** is a Python file created by us for storing reusable code.

For example:

calculator.py

contains calculator functions.

Another program can import it.

SUWEBSOFT FOUNDATION

calculator.py

|

| import

▼

main.py

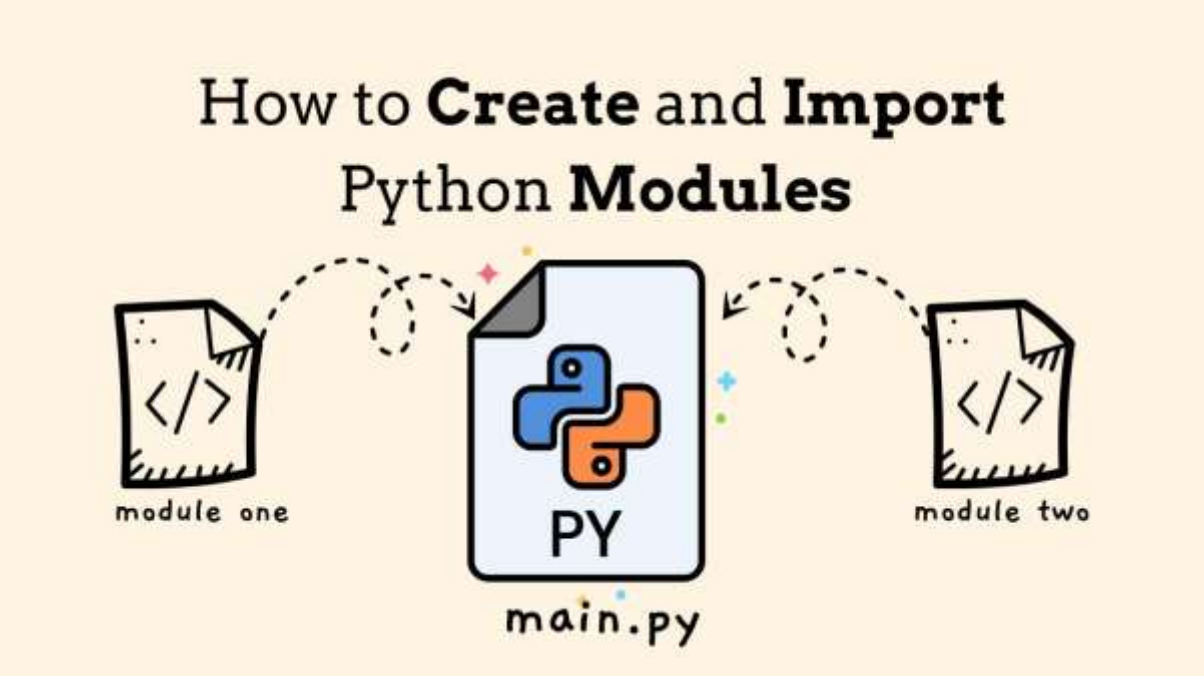
```python
# main.py

import my_module

name = "Alice"
result = my_module.greet(name)
print(result)

x, y = 3, 5
sum_result = my_module.add(x, y)
print(sum_result)
```





12. Practical: Create Your First Custom Module

Step 1 – Create a folder

Create:

ModuleDemo

Inside it create two Python files:

ModuleDemo/

|

├── calculator.py

└── main.py

13. Step 2 – Write calculator.py

```
def add(a, b):
    return a + b
```

3DWEBSOFT FOUND

```
def subtract(a, b):
    return a - b

def multiply(a, b):
    return a * b

def divide(a, b):
    return a / b
```

This is our **custom module**.

14. Step 3 – Write main.py

```
import calculator

print(calculator.add(10, 5))
print(calculator.subtract(10, 5))
print(calculator.multiply(10, 5))
print(calculator.divide(10, 5))
```

Output

```
15
5
50
```

JUWEBSOFT FOUNDATION

2.0

Flow

main.py

```
  |
  | import calculator
  ▼
```

calculator.py

```
  |
  ├── add()
  ├── subtract()
  ├── multiply()
  └── divide()
```

15. Custom Module with Student Functions

Create:

student.py

Write:

```
def welcome(name):
    return "Welcome " + name



def calculate_percentage(m1, m2, m3):
```

3DWEBSOFT FOUNDATION

```
    total = m1 + m2 + m3
    return total / 3
```

Now create:

main.py

```
import student

print(student.welcome("Rahul"))
percentage = student.calculate_percentage(80, 75, 90)
print("Percentage:", percentage)
```

Output

```
Welcome Rahul
Percentage: 81.66666666666667
```

16. from ... import

Instead of:

```
import student

student.welcome("Asha")
```

we can write:

```
from student import welcome

print(welcome("Asha"))
```

This directly imports the required function.

3DWEBSOFT FOUNDATION

17. Import Everything

Python also allows:

from student import *

Then:

print(welcome("Asha"))

However, students should generally prefer **explicit imports**, because importing everything can make larger programs harder to understand.

18. Module Variables

A module can also contain variables.

school.py

```
school_name = "ABC High School"
school_city = "Hyderabad"


def display_school():
    print(school_name)
    print(school_city)
```

main.py

```
import school
print(school.school_name)
print(school.school_city)
school.display_school()
```

3DWEBSOFT FOUNDATION

19. Module with Functions and Variables

```
# marks.py


PASS_MARK = 40


def is_pass(mark):
    return mark >= PASS_MARK
def grade(mark):
    if mark >= 90:
        return "A+"
    elif mark >= 75:
        return "A"
    elif mark >= 60:
        return "B"
    elif mark >= 40:
        return "C"
    else:
        return "F"
```

Use it:

```
import marks


print(marks.PASS_MARK)
print(marks.is_pass(75))
```

3DWEBSOFT FOUNDATION

```
print(marks.grade(85))
```

Output:

```
40
True
A
```

20. What is a Package?

A **package** is a directory used to organize related Python modules.

For example:

```
school/
|
 ├── __init__.py
 ├── students.py
 ├── teachers.py
 └── marks.py
```

Here:

school

↓

Package

students.py

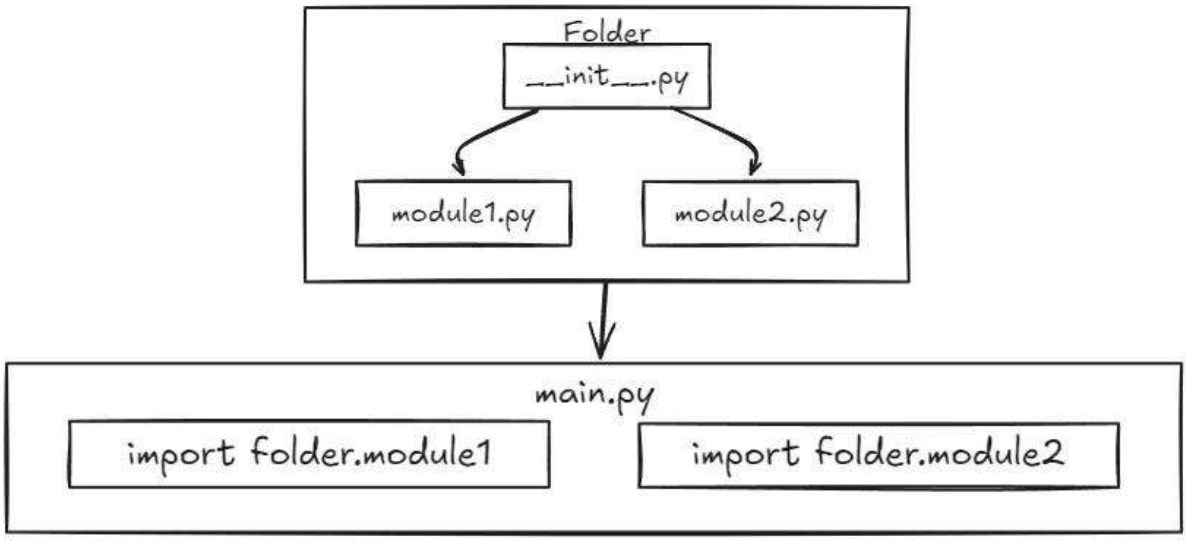
teachers.py

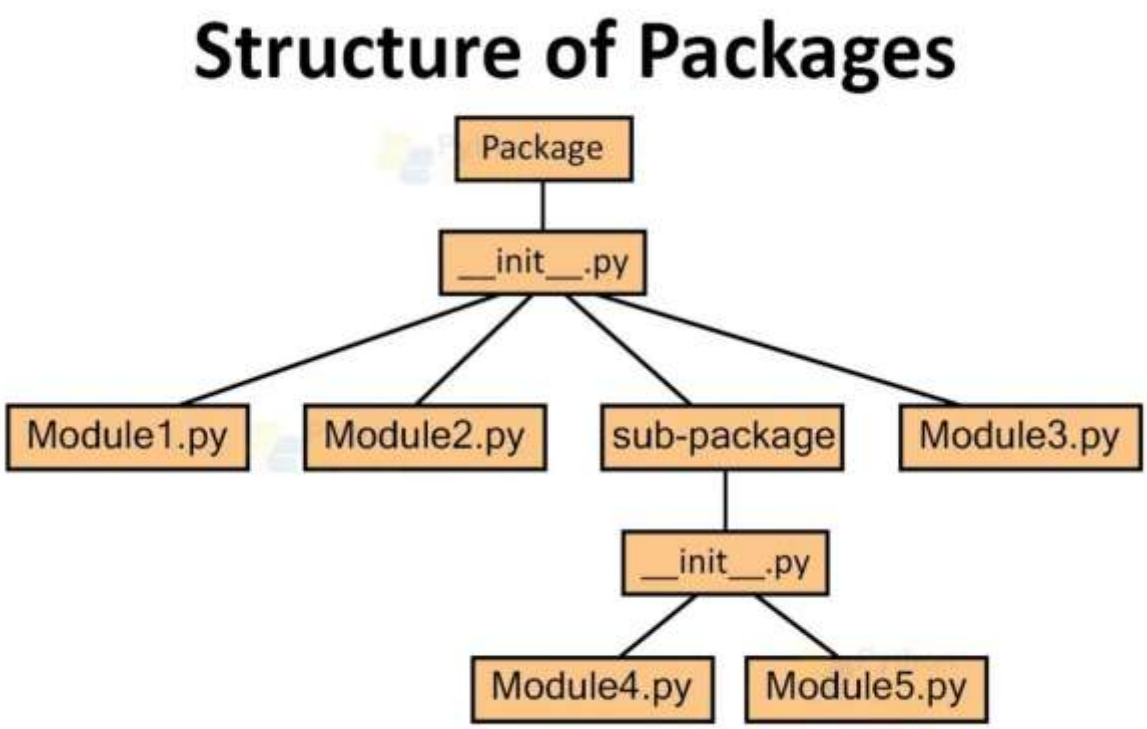
3DWEBSOFT FOUNDATION

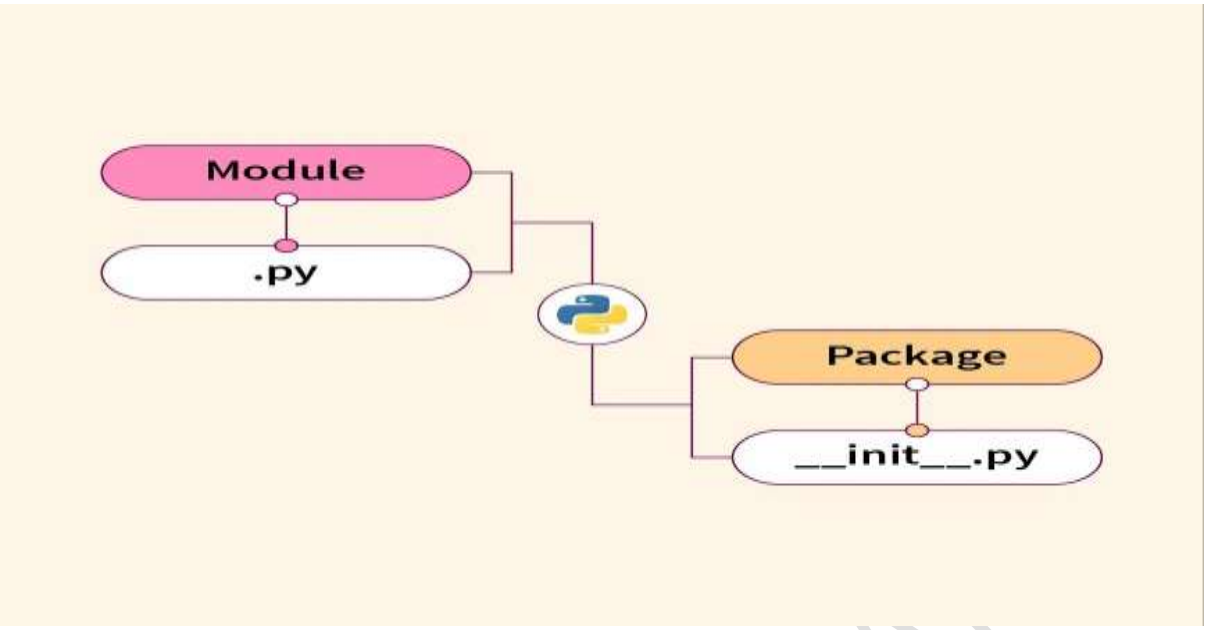
marks.py

↓

Modules







21. Module vs Package

Module	**Package**
Usually a .py file	Usually a directory
Contains Python code	Organizes related modules
Example: marks.py	Example: school/
Used for reusable code	Used for larger organization

Easy way to remember

Module = One Python file

Package = Collection/organization of modules

22. Practical – Create a Custom Package

Create this structure:

3DWEBSOFT FOUNDA

```
SchoolProject/
|
 ├── main.py
|
└── school/
    ├── __init__.py
    ├── students.py
    └── marks.py
```

students.py

```
def welcome(name):
    return "Welcome " + name



def student_type(age):
    if age >= 18:
        return "Adult Student"
    else:
        return "School Student"
```

marks.py

```
def grade(mark):
    if mark >= 90:
```

3DWEBSOFT FOUNDATION

```
        return "A+"
    elif mark >= 75:
        return "A"
    elif mark >= 60:
        return "B"
    elif mark >= 40:
        return "C"
    else:
        return "F"
```

23. Import from the Package

In main.py:

```
from school.students import welcome
from school.marks import grade

print(welcome("Priya"))
print("Grade:", grade(86))
```

Output

```
Welcome Priya
Grade: A
```

24. Understanding the Import Path

Look at:

SUWEBSOFT FOUNDATION

from school.students import welcome

Break it down:

school

↓

Package

students

↓

Module

welcome

↓

Function

So:

Package → Module → Function

25. __name__ == "__main__"

Students should also understand this important concept.

Suppose:

calculator.py

```
def add(a, b):
    return a + b
```

3DWEBSOFT FOUNDATION

```
if __name__ == "__main__":
    print(add(10, 20))
```

When we run:

calculator.py

directly, the code inside:

```
if __name__ == "__main__":
```

runs.

When the file is imported by another program, that block does not run as the module's main program.

26. Why Use __name__ == "__main__"?

It helps us separate:

Reusable module code

from

Code that should run only when the file is executed directly.

Example:

```
def greet():
    print("Hello!")

if __name__ == "__main__":
    greet()
```

3DWEBSOFT FOUNDATION

Practical Session

27. Practical Project – Student Utility Module

Create:

StudentProject/

|

├── main.py

└── student_utils.py

student_utils.py

```
def calculate_total(marks):
    return sum(marks)



def calculate_percentage(marks):
    total = sum(marks)
    return total / len(marks)



def get_result(percentage):
    if percentage >= 40:
        return "PASS"
    else:
        return "FAIL"
```

3DWEBSOFT FOUNDATION

main.py

```
import student_utils

marks = [80, 75, 90, 85, 70]

total = student_utils.calculate_total(marks)

percentage = student_utils.calculate_percentage(marks)

result = student_utils.get_result(percentage)


print("Total:", total)

print("Percentage:", percentage)

print("Result:", result)
```

Output

```
Total: 400

Percentage: 80.0

Result: PASS
```

28. Practical Activity

Activity 1 – Calculator Module

Create:

calculator.py

with:

add()

subtract()

3DWEBSOFT FOUNDATION

multiply()

divide()

Then import it into:

main.py

Test all four functions.

Activity 2 – Temperature Module

Create:

temperature.py

with:

```
def celsius_to_fahrenheit(c):
    return (c * 9/5) + 32



def fahrenheit_to_celsius(f):
    return (f - 32) * 5/9
```

Use it from main.py.

Activity 3 – Student Module

Create:

student.py

with:

```
def calculate_total(marks):
```

3DWEBSOFT FOUNDATION

```
    return sum(marks)


def calculate_average(marks):
    return sum(marks) / len(marks)
```

Import the module and display the results.

29. Mini Project – School Management Package

Create:

```
SchoolManagement/
|
 ├── main.py
|
└── school/
    ├── __init__.py
    ├── student.py
    ├── marks.py
    └── fees.py
```

Student module

```
def display_student(name, age):
    print("Name:", name)
    print("Age:", age)
```

Marks module

3DWEBSOFT FOUNDATION

```
def total(marks):
    return sum(marks)


def average(marks):
    return sum(marks) / len(marks)
```

Fees module

```
def remaining_fee(total_fee, paid_fee):
    return total_fee - paid_fee
```

Main program

```
from school.student import display_student
from school.marks import total, average
from school.fees import remaining_fee

display_student("Arjun", 17)

marks = [80, 75, 90]

print("Total:", total(marks))
print("Average:", average(marks))

print("Remaining Fee:", remaining_fee(30000, 20000))
```

Expected Output

3DWEBSOFT FOUNDATION

Name: Arjun

Age: 17

Total: 245

Average: 81.66666666666667

Remaining Fee: 10000

30. Advantages of Modules and Packages

1. Reusability

Write code once and use it many times.

2. Organization

Large programs can be divided into smaller files.

3. Easy Maintenance

Changes can be made in one module.

4. Easy Debugging

Errors can be isolated more easily.

5. Teamwork

Different programmers can work on different modules.

6. Code Readability

Programs become easier to understand.

31. Common Mistakes

Mistake 1 – Wrong filename

©JWEBSOFT FOUNDATION

If the file is:

calculator.py

write:

import calculator

not:

import Calculator

on systems where case differences matter.

Mistake 2 – Files in different locations

For a simple custom module, keep both files in the same project directory:

Project/

├── main.py

└── calculator.py

Mistake 3 – Function name doesn't exist

If the module contains:

def add(a, b):

then use:

calculator.add(10, 20)

not:

calculator.sum(10, 20)

unless sum() was actually defined.

32. Quick Revision Table

3DWEBSOFT FOUNDATION

Concept	Meaning
Module	Python file containing reusable code
Package	Collection/organization of related modules
import	Imports a module
from	Imports selected items
as	Creates an alias
__init__.py	Package initialization file commonly used in package structures
Custom module	Module created by the programmer
__name__	Special variable containing the module's name

33. Exam Questions

Short Answer

1. What is a Python module?
2. What is a package?
3. Why are modules useful?
4. What is the purpose of import?
5. What is a custom module?
6. What is an alias?
7. What is the difference between a module and a package?
8. What is the purpose of __name__ == "__main__"?

3DVEBSOFT FOUNDATION

Programming Questions

Q1. Create a module named calculator.py with addition and subtraction functions.

Q2. Create a module named marks.py to calculate total and average marks.

Q3. Create a package containing separate modules for students and marks.

Q4. Import a function from a custom module using:

from module_name import function_name

34. Classroom Challenge

Give students this requirement:

"Create your own Python utility module containing at least 3 functions and use it from another Python file."

Students can choose:

- Calculator
- Student marks
- Temperature converter
- Shopping bill
- Currency calculator
- Geometry calculator

Challenge Structure

MyProject/

3DWEBSOFT FOUNDATION

```
|
├── main.py
└── my_module.py
```

Students must demonstrate:

Create module

↓

Write functions

↓

Import module

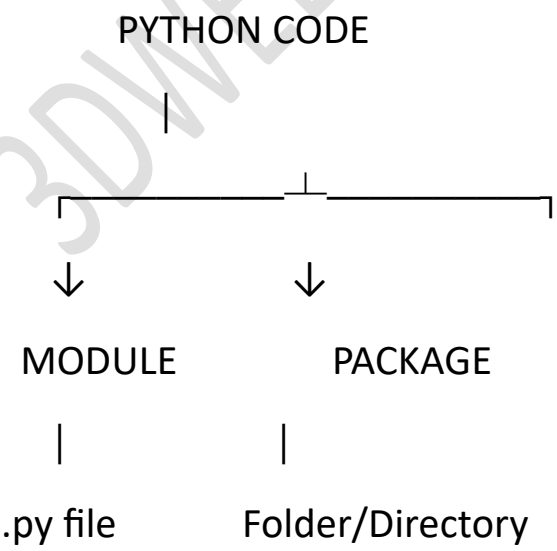
↓

Call functions

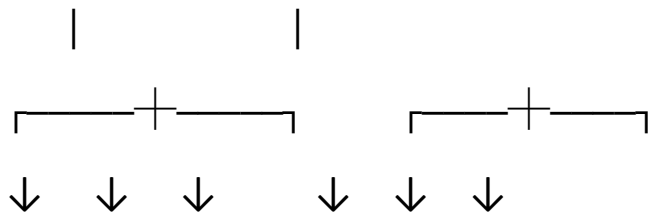
↓

Display output

35. Final Concept Map



3DWEBSOFT FOUNDATION



import

Main Program

Student Takeaway

A module is a Python file containing reusable code. A package organizes related modules. By creating custom modules, programmers can divide large programs into smaller, reusable and manageable parts.

3DWEBSOFT FOUNDATION